SENTINELOPS

Self-Healing Infrastructure & Intelligent Incident Recovery Platform

1. Introduction

Service availability is a core operational concern: a failed service can interrupt users and require prompt intervention. SentinelOps is a DevOps proof of concept that connects monitoring, alert routing, and automated recovery. It uses an NGINX web service as the monitored target and combines Prometheus, Alertmanager, Blackbox Exporter, a Flask-based recovery controller, and Ansible in a Docker Compose environment.

2. Abstract

SentinelOps demonstrates a service-availability monitoring-to-recovery workflow. Blackbox Exporter probes the NGINX HTTP endpoint and exposes probe metrics to Prometheus. Prometheus evaluates configured alert rules; when a service-down condition persists for the configured duration, it fires an alert. Alertmanager forwards matching alerts to the recovery controller, which can invoke the Ansible recovery playbook and record recovery activity. A Flask dashboard presents service status, alert information, and recovery events. Verification during implementation showed HTTP 200 responses from both the SentinelOps API health endpoint and the NGINX target.

3. Tools Used

Tool / Component	Role in the project
Docker Compose	Runs the application, monitored target, monitoring components, and recovery services as containers.
Prometheus	Collects metrics and evaluates service-availability alert rules.
Blackbox Exporter	Probes the NGINX HTTP endpoint and reports probe success or failure.
Alertmanager	Routes firing alerts to the recovery controller webhook.
Python / Flask	Provides the dashboard/API and recovery-controller endpoints.
Ansible	Executes the configured NGINX recovery playbook.
NGINX	Acts as the sample service monitored by the platform.

4. Steps Involved

1. **Containerized deployment:** Organized the dashboard/API, NGINX target, Prometheus, Alertmanager, Blackbox Exporter, and recovery controller through Docker Compose.
2. **Monitoring configuration:** Configured Blackbox Exporter to probe NGINX and Prometheus to scrape probe results.
3. **Alert definition:** Added an NGINX service-down rule based on probe success and an API-down rule based on target availability.

IMPLEMENTATION & VALIDATION

4. Steps Involved (continued)

4. **Alert routing:** Configured Alertmanager to forward matching alerts to the recovery controller using its internal webhook endpoint.
5. **Recovery automation:** Connected the recovery controller to the Ansible playbook. Recorded recovery output included a successful Ansible recap with *failed=0*; the NGINX endpoint was subsequently verified as responding.
6. **Operational visibility:** Exposed dashboard information for service health, alerts, and recovery events.
7. **Verification and evidence:** Checked the API health endpoint and NGINX target; both returned HTTP 200. Six screenshots document the dashboard, Prometheus rules, Alertmanager configuration, recovery events, recovery logs, and NGINX verification.

Validation Summary

Check	Observed result
SentinelOps API health endpoint	HTTP 200; response reported the API as healthy.
NGINX target endpoint	HTTP 200; default NGINX page returned.
Recovery evidence	Recovery log showed an Ansible recap with failed=0; the endpoint was later verified responding.
Evidence screenshots	Six screenshots are present in the project's evidence directory.

> **Scope and limitations:** This version demonstrates service-availability alerts and NGINX recovery. CPU usage alerting above 90% is not implemented. This is a proof of concept; production use would require additional hardening, access controls, and secret management.

5. Conclusion

SentinelOps integrates open-source DevOps tools into a practical monitoring and recovery pipeline: probe a service, detect a failure, route an alert, invoke Ansible recovery, and review operational evidence in a dashboard. The implementation provides a foundation for further work, including resource-based alerts, expanded automated tests, recovery policy controls, and production-grade security.